



김지관

Software Engineer

☎ 010-9895-5140

✉ zc149@naver.com

🐙 [GitHub](#)

핵심 역량

새로운 기술을 도입할 때 충분한 검토와 PoC를 통해 적합성과 안정성을 검증한 후 운영 환경에 적용하는 것을 중요하게 생각합니다.

주어진 기능을 구현하는 데 그치지 않고, 실제 사용자의 관점에서 필요한 기능과 개선 방향을 주도적으로 제안해 왔습니다.

구축 이후에도 지속적으로 피드백을 반영하며 운영 효율성과 사용자 경험을 개선하고, 서비스에 적합한 형태로 완성도를 높여가고 있습니다.

기술 스택

- Backend: Spring, Node.js, Go
- Frontend: React, TypeScript
- Infra/DevOps: Kubernetes, Helm, Argo CD, Terraform, AWS
- DB: MySQL, Oracle, PostgreSQL

오픈소스 기여

argo-cd: [argoproj/argo-cd#25590](#)

Argo CD 파이프라인 개선 과정에서 약 70여 개 서비스의 Application 설정을 수정해야 했으나, Dashboard에서 annotation 기반 조회 기능이 없어 각 Application을 개별적으로 탐색해야 하는 비효율이 발생했습니다.

문제를 해결하기 위해 Argo CD 저장소에 Issue를 등록하고 annotation 기반 조회 기능을 구현하여 프로젝트에 기여했습니다.

Backstage Plugin: [backstage/backstage#32216](#)

Backstage 기반 IDP 구축 과정에서 서비스 단위 DORA Metrics 시각화 플러그인을 개발하고 Backstage Plugin Marketplace에 배포했습니다.

주당 약 50~100회 다운로드가 발생하며 실제 사용자들이 활용 중이고, 사용자 Issue 및 PR이 이어지면서 Backstage 생태계에 기여했습니다. 이를 기반으로 지속적인 기능 개선 및 유지보수를 진행하고 있습니다.

terrateam: terrateamio/terrateam#1058

Terrateam 사용 중 발견한 UI 이슈를 재현하고 수정한 기여입니다. 기능 확장보다는 사용자 경험을 개선하는 버그 수정 성격의 PR입니다.

경력사항

● 씨제이올리브영주식회사

- 기간: 2025.09 - 2026.09
- 직무: DevOps
- 직급: 인턴
- 주요 프로젝트:
 - 내부 개발자 플랫폼(IDP) 구축
 - EKS 환경에 NodeLocal DNS 도입
 - EKS 신규 클러스터 구축 및 서비스 마이그레이션
 - 오픈소스 기반 Incident Management 시스템 내제화
 - Gradle 빌드 캐시 최적화
 - ECR Multi-Region 및 비용 최적화
- 장애 분석 및 트러블슈팅:
 - Aurora PostgreSQL CDC Replication Slot 장애 분석
 - Istio Ambient Waypoint 503 장애 분석 및 재발 방지

● 주식회사스노우온카드

- 기간: 2024.12 - 2025.09
- 직무: 서버개발
- 직급: 대리
- 주요 프로젝트:
 - Token 기반 비접촉 결제 솔루션 구축 (HCE / Scan to Pay)
 - 간편결제 통합 인터페이스 구축(SI)

프로젝트

● 씨제이올리브영주식회사

내부 개발자 플랫폼(IDP) 구축

- 기간: 2025.09 - 2026.02
- 역할: Backstage 기반 IDP 개발 및 DevOps 플랫폼 구축
- 기술: Backstage, React, Node.js, TypeScript, Kubernetes, Helm, Argo CD

배경

기존 SaaS 기반 개발자 포털은 조직 요구사항에 맞춘 기능 확장에 한계가 있었고, 신규 서비스 생성 및 배포 과정에서 개발자가 GitOps Repository의 Kubernetes Values와 배포 설정을 직접 작성해야 했습니다. 이로 인해 프로젝트 초기 설정 편차가 발생하고, 신규 서비스 배포까지 반복적인 수작업이 필요했습니다.

수행 내용

- Backstage 기반 내부 개발자 플랫폼을 구축하여 기존 Port.io 기반 환경을 대체
- Scaffolder Template으로 Repository 생성, Kubernetes Values, Argo CD 배포 설정 생성 과정을 자동화
- 공통 베이스 Repository를 Template으로 등록하여 프로젝트 생성 및 초기 설정 표준화
- 기존 Repository를 Backstage Catalog로 마이그레이션하여 서비스 메타데이터와 Repository를 통합 관리
- Argo CD, k6 Operator, Spring Batch 등 주요 개발·운영 도구를 Backstage Plugin으로 통합

성과

- 신규 서비스 생성부터 배포까지 걸리는 시간을 약 5분 수준으로 단축
- Repository 생성 경로를 Backstage로 일원화하여 표준화된 개발 프로세스 적용
- 개발자가 여러 운영 도구를 개별적으로 접근하지 않고 하나의 포털에서 배포, 성능 테스트, 배치 관리 업무를 수행할 수 있도록 DX 개선
- 서비스 생애주기와 메타데이터를 Backstage Catalog에서 일관되게 관리할 수 있는 기반 구축

EKS 환경에 NodeLocal DNS 도입

- 기간: 2026.04
- 역할: EKS DNS 안정성 개선 및 클러스터 운영 안정화
- 기술: EKS, Kubernetes, CoreDNS, NodeLocal DNSCache

배경

EKS 환경에서 AWS Managed Add-on CoreDNS의 롤링 업데이트 또는 일시적인 장애가 발생하면 CoreDNS 응답 지연과 DNS Lookup 실패로 인해 Pod 간 통신에 영향을 줄 수 있었습니다. CoreDNS 의존도를 낮추고 Node 단위에서 안정적으로 DNS 요청을 처리할 수 있는 구조가 필요했습니다.

수행 내용

- EKS 환경에 NodeLocal DNS를 도입하여 DNS 요청을 Node 단위 Local Cache에서 우선 처리하도록 구성
- 기존에 조회한 DNS 레코드는 Local Cache에서 응답하도록 설정하여 CoreDNS 장애 또는 업데이트 상황의 서비스 영향 최소화
- CoreDNS로 전달되는 DNS 트래픽을 줄일 수 있도록 Node 단위 DNS Proxy 구조 구성

성과

- CoreDNS 장애 또는 롤링 업데이트 시 DNS Lookup 실패로 인한 Pod 간 통신 영향 완화
- DNS 요청을 Node 단위로 분산하여 CoreDNS 부하 감소
- 클러스터 DNS 응답 안정성과 서비스 지속성 개선

EKS 신규 클러스터 구축 및 서비스 마이그레이션

- 기간: 2026.04
- 역할: 신규 EKS 클러스터 구축, 네트워크 구조 개선, 서비스 마이그레이션
- 기술: Terraform, EKS, VPC, Secondary CIDR, Route53, AWS

배경

기존 운영 클러스터는 네트워크 구조를 직접 변경하기 어려웠고, Pod IP 확장성 확보를 위해 Secondary IP 대역이 적용된 신규 EKS 클러스터가 필요했습니다. 또한 환경별로 클러스터 구성과 Terraform 코드가 다르게 관리되어 운영 일관성과 유지보수성 개선이 필요했습니다.

수행 내용

- Secondary IP 대역이 적용된 신규 EKS 클러스터를 Terraform 기반으로 구축
- 신규 클러스터 검증 후 기존 클러스터에서 운영 중인 서비스를 단계적으로 이전
- Route53 Weighted Routing을 활용하여 DNS 가중치를 조정하며 트래픽을 신규 클러스터로 전환
- Dev, Stg, Prd 순서로 마이그레이션을 진행하여 운영 서비스 영향을 최소화
- 환경별로 상이했던 클러스터 구성과 Terraform 코드를 표준화

성과

- Secondary IP 기반 네트워크 구조를 적용하여 Pod IP 부족 문제를 해결하고 클러스터 확장성 확보
- DNS 가중치 기반 단계적 트래픽 전환으로 서비스 영향 없이 마이그레이션 수행
- EKS 클러스터 구성과 IaC 관리 방식을 표준화하여 운영 효율성과 유지보수성 향상

오픈소스 기반 Incident Management 시스템 내제화

- 기간: 2026.05
- 역할: Incident Management 솔루션 검토, On-Call 및 Alert 대응 프로세스 구축
- 기술: OneUptime, Incident Management, On-Call, Alerting, Slack

배경

개발 인원 증가에 따라 PagerDuty 라이선스 비용 부담이 커지고 있었고, Alert 발생부터 Incident 대응까지 자체 운영 가능한 오픈소스 기반 Incident Management 및 On-Call 시스템이 필요했습니다.

수행 내용

- 오픈소스 Incident Management 솔루션의 On-Call, Escalation, Alert 연계, 운영 편의성을 비교·검증
- PagerDuty 대체 솔루션으로 OneUptime을 선정하고 신규 프로젝트에 적용
- Alert 발생부터 Incident 생성, 담당자 알림 및 대응까지 이어지는 운영 프로세스 구축
- Alert Severity에 따라 On-Call 담당자를 자동 호출하고 Incident를 자동 생성하도록 구성
- Incident 발생 시 Slack 전용 채널을 자동 생성하고 담당자 및 관련자를 자동 초대하는 협업 프로세스 구현

성과

- PagerDuty를 대체할 수 있는 오픈소스 기반 Incident Management 운영 환경 마련
- Alert Severity 기반 Escalation과 Incident 자동 생성으로 장애 대응 프로세스 표준화
- 개발자뿐만 아니라 비개발 직군도 Incident 대응 과정에 참여할 수 있는 Slack 기반 협업 체계 구축

Gradle 빌드 캐시 최적화

- 기간: 2026.05
- 역할: CI/CD 빌드 캐시 구조 개선 및 빌드 시간 최적화
- 기술: GitHub Actions ARC, Kubernetes, AWS, S3, EFS, Gradle

배경

여러 Repository의 Gradle 캐시가 하나의 EFS 경로에 누적되면서 캐시 크기와 탐색 시간이 증가하고 있었습니다. 이로 인해 EFS I/O 병목과 비용 부담이 발생했고, Repository 단위로 캐시를 분리하면서도 서비스별 Workflow 수정 범위를 최소화할 수 있는 공통 캐시 구조가 필요했습니다.

수행 내용

- 기존 EFS 기반 Gradle 캐시 저장 구조를 Repository별 S3 경로 기반 구조로 전환
- GitHub Actions Runner Hook을 활용하여 Job 시작 시 캐시 복원, Job 종료 시 캐시 저장 과정을 자동화
- Runner Pod 생명주기와 캐시 동기화 시점을 분리하여 Workflow 변경 없이 Runner 레벨에서 공통 캐시 정책 적용
- 서비스별 CI Workflow에 반복 로직을 추가하지 않고 ARC Runner 기반으로 캐시 라이프사이클 제어

성과

- Gradle 빌드 시간을 약 39% 단축
- EFS I/O 병목을 완화하고 Gradle 캐시 저장 구조의 운영 비용 부담 감소
- Repository별 캐시 분리 구조를 적용하여 캐시 관리 안정성과 재사용성 개선

ECR Multi-Region 및 비용 최적화

- 기간: 2026.07
- 역할: ECR Multi-Region 구조 개선 및 Container Image 비용 최적화
- 기술: Amazon ECR, ECR Pull Through Cache, ECR Lifecycle Policy, EKS, CI/CD

배경

Virginia EKS Cluster에서 Seoul 리전의 ECR Container Image를 직접 Pull하면서 Cross-Region Data Transfer 비용이 발생하고 있었습니다. 또한 CI/CD 과정에서 Container Image가 지속적으로 누적되면서 ECR Storage 비용이 증가하고 있어, 운영 배포 이력은 보호하면서 불필요한 이미지를 정리할 수 있는 구조가 필요했습니다.

수행 내용

- Virginia 리전에 ECR Pull Through Cache를 구성하여 반복적인 Image Pull을 리전 내부에서 처리하도록 개선
- Production 배포 시 prd-YYYYMMDD-commitHash 형식의 Tag를 추가하여 실제 운영 배포 Image와 Rollback 이력 보호
- ECR Lifecycle Policy를 적용하여 운영 배포 이미지를 제외한 30일 이상 경과 Image 자동 삭제 구성
- Cross-Region Data Transfer 비용과 ECR Storage 비용을 함께 줄일 수 있도록 Multi-Region Image Pull 구조와 보관 정책 개선

성과

- Virginia EKS Cluster의 반복적인 Image Pull을 리전 내부에서 처리하도록 개선하여 Cross-Region Data Transfer 비용 절감
- 운영 배포 Image 및 Rollback 이력을 보호하면서 불필요한 CI/CD Image 누적 문제 완화
- ECR 관련 월 비용을 약 \$300에서 \$200 수준으로 낮춰 약 33% 절감

주식회사스노우온카드

Token 기반 비접촉 결제 솔루션 구축 (HCE / Scan to Pay)

- 기간: 2025.06 - 2025.07
- 직무: 서버개발
- 기술스택: Spring, MySQL
- 설명: Visa, Mastercard 등 국제 브랜드의 토큰 기반 결제 프로세스를 구현하고, HCE 및 QR 기반 Scan to Pay 기능을 포함한 TLCM 솔루션 개발

주요 업무 및 성과

- EMVCo 기반 Visa / Mastercard 결제 흐름 분석 및 토큰화 프로세스 구현 (Provisioning, Payment, Lifecycle 관리)
- Spring 기반 백엔드 모듈 설계 및 구현 (TSP 연동, HCE 카드 생성, QR 결제 요청 처리 등)
- 카드사 및 브랜드사 요구사항에 맞춘 TLCM (Transaction Life Cycle Management) 기능 구현
- 테스트 시뮬레이션 환경 구축 (APDU 커맨드/응답 처리)

간편결제 통합 인터페이스 구축(SI)

- 기간: 2025.02 - 2025.05
- 직무: 서버개발
- 기술스택: React, TypeScript, Spring, Oracle
- 설명: PG사 간편결제 등록·승인·취소 API를 통합 관리하는 백엔드 인터페이스와 어드민 시스템 구축

주요 업무 및 성과

- 주요 PG사 간편결제 연동 API 설계 및 구현 (Spring 기반 서버 개발)
- TO-BE 정규화 DB 스키마 재설계 및 데이터 마이그레이션
- React 기반 어드민 대시보드 개발 (결제 통계: 건수, 승인 실패율 등 실시간 시각화)

장애 분석 및 트러블슈팅

🔴 Aurora PostgreSQL CDC Replication Slot 장애 분석

- 기간: 2026.07
- 역할: Aurora PostgreSQL I/O 폭증 원인 분석 및 CDC 운영 안정성 개선
- 기술: Aurora PostgreSQL, PostgreSQL MVCC, Debezium, Kafka Connect, Logical Replication Slot

배경

DEV 환경의 Aurora PostgreSQL에서 평소 대비 Read IOPS와 CPU 사용률이 비정상적으로 증가하고, Autovacuum이 반복 수행되는 현상이 발생했습니다. Dead Tuple이 빠르게 증가하고 있었지만 Autovacuum이 정상적으로 정리하지 못했고, 최대 CPU 사용률이 99%까지 상승하면서 Aurora I/O 비용도 함께 증가했습니다.

수행 내용

- `pg_replication_slots`, `transaction xmin`, Dead Tuple 상태를 확인하여 Debezium / Kafka Connect CDC 환경의 Logical Replication Slot 영향 분석
- 사용되지 않는 Replication Slot이 오래된 `xmin`을 유지하면서 PostgreSQL이 Dead Tuple을 제거하지 못하는 구조를 규명
- Dead Tuple 누적으로 인해 Autovacuum이 반복적으로 대량 Page를 읽고, Aurora Read IOPS와 CPU 사용률이 폭증하는 흐름 분석
- 미사용 Replication Slot을 식별 및 제거하고, Autovacuum / Dead Tuple 상태를 지속 모니터링

성과

- Debezium CDC Replication Slot, PostgreSQL `xmin`, Dead Tuple, Autovacuum, Aurora I/O 비용 증가로 이어지는 장애 구조를 규명
- 미사용 Replication Slot 제거 이후 오래된 `xmin`이 해소되면서 Dead Tuple 정리와 Autovacuum 동작 정상화

🔴 Istio Ambient Waypoint 503 장애 분석 및 재발 방지

- 기간: 2026.08
- 역할: 서비스 간 간헐적 503 원인 분석 및 Connection Pool 정책 표준화
- 기술: Istio Ambient, Waypoint, Envoy, DestinationRule, Spring Boot, Embedded Tomcat, HTTP Keep-Alive

배경

Production 환경에서 Spring Boot 서비스 간 호출 중 정상 요청 대부분은 성공하지만 일부 요청에서 간헐적인 HTTP 503 오류가 발생했습니다. 지속적인 장애가 아니라 특정 시점에만 발생했기 때문에 Application 로직이나 단순 Network 장애만으로 원인을 특정하기 어려운 상황이었습니다.

수행 내용

- Istio Ambient 환경의 zTunnel / Waypoint 구간을 분리하여 503 발생 지점 분석
- Envoy Connection Pool과 Tomcat Keep-Alive 정책을 비교해 Connection Reuse Race Condition을 원인으로 규명
- HTTPRoute가 아닌 DestinationRule을 통해 idleTimeout, maxRequestsPerConnection 기반 Upstream Connection 정책 적용
- Tomcat maxKeepAliveRequests보다 낮은 maxRequestsPerConnection: 90을 설정하여 Envoy가 먼저 Connection을 교체하도록 구성
- Base Chart에 DestinationRule 템플릿과 Values Override 인터페이스를 추가하여 서비스별 Keep-Alive 설정에 대응

성과

- Istio Ambient Waypoint와 Tomcat의 서로 다른 Connection Lifecycle 정책에서 발생하는 간헐적 503 원인을 규명
- DestinationRule 기반 Connection Pool 정책을 적용하여 Waypoint가 Tomcat에서 이미 종료한 Connection을 재사용하며 발생하던 503 Race Condition 방어
- 개별 서비스 장애를 공통 기술 스택의 안정성 이슈로 확장하고, 동일 구조를 사용하는 서비스 전체에 적용 가능한 표준 설정 마련

자격증

🎯 AWS Certified Solutions Architect - Associate (SAA)

- 2026.02 / AWS

🎯 리눅스마스터 2급

- 2024.09 / 한국정보통신인력개발센터

🎯 정보처리기사

- 2024.09 / 한국산업인력공단

🎯 SQL개발자(SQLD)

- 2024.06 / 한국데이터베이스진흥센터

학력

● 인하대학교

- 2014.03 - 2021.02 (졸업)
- 항공우주공학과 학사